TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN

HỌC PHẦN: CHUYÊN ĐỀ TỐT NGHIỆP 1
BÀI TẬP 3 – KIỂM THỬ VÀ ĐÓNG GÓI PROTOTYPE

Đề tài: Hệ thống Smart CRM – Mekong Mobile
Mã luồng nghiệp vụ: L2
Tiếp nhận và phân loại yêu cầu bảo hành (Service Desk)
Track: SE (Software Engineering)

Họ tên: Thái An Quốc
MSSV: 2374802012635
Lớp: 261_71ITGR40203_06
Giảng viên: TS. Nguyễn Trí Hải
Ngày nộp: 12/ 11/ 2026
Link GitHub repo: https://github.com/anquoc2024/cdtn1-thaianquoc-2
Tên tag: v1.0 –
https://github.com/anquoc2024/cdtn1-thaianquoc-2/releases/tag/v1.0
Link bản deploy thử: https://smartcrm-app.onrender.com

TP Hồ Chí Minh – 2026

MỤC LỤC

MỤC 1. TEST PLAN RÚT GỌN 3
1. Phạm vi kiểm thử và phần chủ ý KHÔNG kiểm 3
2. Loại test và số lượng mỗi tầng 3
3. Tiêu chí thành công ĐO ĐƯỢC (đặt trước khi chạy) 4
4. Môi trường và dữ liệu thử 4
MỤC 2. TEST REPORT 5
Đối chiếu năm loại test BT3 8
MỤC 3. BUG LOG 9
BUG-01 — Quản lý trung tâm dùng được chức năng của Nhân viên tiếp nhận 9
BUG-02 — Clone sạch không dựng được hệ thống vì thiếu CustomerRowNormalizer 9
BUG-03 — API yêu cầu bảo hành không dùng được: thiếu route và controller 10
BUG-04 — Thẻ thống kê bị cắt dòng mốc so sánh 10
BUG-05 — 3 test sửa / xóa yêu cầu hỏng do biến @request 10
BUG-06 — Thư viện có lỗ hổng bảo mật đã công bố 11
BUG-07 — Gửi JSON hỏng bị trả 500 thay vì 400 11
BUG-08 — Mật khẩu demo bị in ra log container 12
BUG-09 — Clone trên Windows thì container app không khởi động 12
MỤC 4. DEPLOYMENT GUIDE 14
1. Requirements 14
2. Setup steps 14
3. Environment variables 14
4. Sample data 15
5. Verify it works 16
6.Troubleshooting 16
7. Trial deployment on Render 17
MỤC 5. CHECKLIST BẢO MẬT 7 MỤC ĐÃ TỰ ĐÁNH GIÁ 19
MỤC 6. TÀI LIỆU KỸ THUẬT THEO CHUYÊN NGÀNH – API DOCS (SE) 20
Real examples (from docs/api-examples.md, shortened: "…" marks fields left out; tokens replaced by placeholders) 22
MỤC 7. ẢNH CHỤP MINH CHỨNG 24
7.1. Clone theo tag v1.0 vào thư mục trống 24
7.2. Hệ thống lên được bằng hai lệnh, có dữ liệu mẫu 24
7.3. Bộ kiểm thử PASS bằng một lệnh 24
7.4. Bản deploy thử (điểm cộng) 25
PHỤ LỤC. BẢNG KHAI BÁO SỬ DỤNG CÔNG CỤ AI (GIAI ĐOẠN 3) 26

MỤC 1. TEST PLAN RÚT GỌN

1. Phạm vi kiểm thử và phần chủ ý KHÔNG kiểm

Kiểm: đăng nhập / đăng xuất và phân quyền theo vai trò (NFR05, TD-24, TD-29) · tra cứu khách hàng (UC01, UC08) · thêm khách hàng · tạo yêu cầu bảo hành (BR01–BR06) · danh sách và lọc quá hạn (BR10) · đổi trạng thái (BR08, BR09) · sửa / xóa yêu cầu (TD-29) · nạp lại dữ liệu mẫu không nhân đôi (TD-28) · ràng buộc dữ liệu BR11–BR14.

Cố ý KHÔNG kiểm (và lý do):

- Hiệu năng, tải đồng thời: thuộc Chuyên đề tốt nghiệp 2.
- Kiểm thử giao diện bằng trình duyệt tự động (Selenium / Capybara): chậm và dễ vỡ; giao diện đã được kiểm ở mức HTML bằng integration test (src/test/integration/*).
- Kiểm soát truy cập theo CHỦ SỞ HỮU từng bản ghi: luồng L2 không có tài khoản khách hàng, Nhân viên tiếp nhận dùng chung một hàng đợi nên quyền được kiểm theo VAI TRÒ (TD-29); test bảo mật kiểm vai trò Quản lý bị chặn (TC-11).
- service_centers.csv: thiết kế chưa có bảng trung tâm (TD-28).

2. Loại test và số lượng mỗi tầng

- Bộ test tự động: **131 test**, chạy bằng MỘT lệnh bin/rails test (thư mục src, hoặc trong container theo docs/deployment-guide.md): unit / service src/test/services/ (máy tính hạn cam kết, sinh mã, làm sạch dữ liệu, máy trạng thái, CRUD service) · integration src/test/integration/ (controller + router + CSDL thật, gồm API).
- Test Report: **16 test case** chạy thật — 12 integration (TC-01–04, 07–11, 13–15) · 2 system / API qua curl vào server thật (TC-05, TC-06) · 1 service + CSDL (TC-12) · 1 integration + API thật (TC-16).
- Đủ 5 loại: luồng đúng ×6 (TC-01, 03, 07, 09, 14, 15) · giá trị biên ×2 (TC-05 / TC-06, hai bên ranh giới SĐT 10 chữ số) · dữ liệu sai ×2 (TC-02, TC-08) · bảo mật ×3 (TC-11 A01, TC-13 A07, TC-16 A05) · tái lập ×1 (TC-12); thêm luồng ngoại lệ UC08 (TC-04) và vi phạm BR08 (TC-10).
- Kỹ thuật chọn dữ liệu: phân vùng tương đương (SĐT hợp lệ / sai độ dài; yêu cầu đủ / thiếu trường) và giá trị biên. Kết quả mong đợi kiểm cả mã trạng thái, thông báo lỗi VÀ tác dụng phụ (đếm bản ghi trước / sau).

Truy vết — mỗi quy tắc nghiệp vụ đã hiện thực có ít nhất một test:

Quy tắc (docs/srs.md)	Test case trong Test Report	Test tự động khác trong bin/rails test
BR01 Xác định khách hàng trước khi tạo yêu cầu	TC-03, TC-06, TC-07	customer_service_test.rb:55
BR02 Thông tin thiết bị đủ, ngày mua không ở tương lai	TC-08	warranty_request_updater_test.rb:43
BR03 Có nhóm sự cố và mức ưu tiên	TC-07, TC-08	issue_category_service_test.rb:25
BR04 Thiếu / sai thông tin thì không tạo	TC-08	—
BR05 Mã BH-YYYY-NNNNNN, trạng thái đầu Mới tiếp nhận	TC-07	request_code_generator_test.rb
BR06 Hạn cam kết = lúc tạo + SLA	TC-07, TC-14	sla_deadline_calculator_test.rb, warranty_request_updater_test.rb:22
BR07 Không tìm thấy khách thì không cho tạo	TC-04	—
BR08 Chuyển trạng thái đúng thứ tự	TC-10	warranty_request_status_changer_test.rb:20

Quy tắc (docs/srs.md)	Test case trong Test Report	Test tự động khác trong bin/rails test
BR09 Hủy phải có lý do; ghi lịch sử	—	warranty_request_status_changer_test.rb:39, api_warranty_requests_test.rb:115
BR10 Quá hạn tính khi truy vấn	TC-09	warranty_request_status_changer_test.rb:63
BR11 Một serial chỉ thuộc một khách	—	warranty_request_creation_test.rb:63, api_warranty_requests_test.rb:94
BR12 Mã khách hàng tự sinh	TC-05	customer_service_test.rb:10
BR13 Không xóa khách đã có thiết bị	—	customer_service_test.rb:63
BR14 Không xóa nhóm sự cố đang dùng	—	issue_category_service_test.rb:32
NFR05 / TD-24 Đăng nhập, phiên	TC-01, TC-02, TC-13	api_sessions_test.rb
TD-29 Phân quyền vai trò (OWASP A01)	TC-11, TC-14, TC-15	authorization_test.rb

3. Tiêu chí thành công ĐO ĐƯỢC (đặt trước khi chạy)

- 16/16 test case có kết quả thực tế ghi bằng số liệu cụ thể và ảnh docs/screenshots/TC-xx.png.
- Mỗi quy tắc BR01–BR14 có ≥ 1 test (bảng truy vết ở mục 2).
- bin/rails test PASS toàn bộ (0 failures, 0 errors) cả trên máy lẫn trong container.
- Không còn bug Critical đang mở; bug Major còn lại (nếu có) ghi lý do và hướng xử lý trong docs/bug-log.md.

4. Môi trường và dữ liệu thử

- Máy phát triển: Windows + WSL 2 (Ubuntu), Ruby 4.0.5, Rails 8.1, PostgreSQL 18. Gói Docker: docker compose với PostgreSQL 18 và app Ruby 4.0.5 (docs/deployment-guide.md).
- Test tự động: CSDL test riêng (src_test), mỗi test tự dựng dữ liệu trong setup và chạy trong transaction được rollback — không phụ thuộc dữ liệu có sẵn trên máy; chạy song song thì mỗi tiến trình một CSDL riêng.
- Test thủ công qua API (TC-05, TC-06, TC-12, TC-16): bin/rails db:reset nạp bộ dữ liệu L2 trong data/mau/ và 2 tài khoản demo, rồi bin/rails server. Dọn: chạy lại bin/rails db:reset.
- Chạy từng test case: bash scripts/tc.sh TC-05; chạy và chụp cả 16: bash scripts/chup-tc.sh.

MỤC 2. TEST REPORT

16 test case trình bày **theo chức năng** người dùng thấy: đăng nhập / đăng xuất, tra cứu khách hàng, thêm khách hàng, tạo yêu cầu, danh sách yêu cầu, cập nhật trạng thái, sửa / xóa yêu cầu, phân quyền, nạp dữ liệu. Chạy lại bất kỳ test case nào: bash scripts/tc.sh TC-xx. Ảnh chụp từng lần chạy: docs/screenshots/TC-xx.png.

Tổng kết: đã chạy thật 16 / 16 · PASS 16 · FAIL 0 · Bộ test tự động (bin/rails test): 129 runs, 578 assertions, 0 failures, 0 errors (lúc chạy 16 TC, commit 47593cf); bản nộp v1.0 có thêm 2 test hồi quy của BUG-07, BUG-08: 131 runs, 591 assertions, 0 failures, 0 errors

Nhật ký chữ của lần chạy: docs/tc-ket-qua.txt (sinh bởi bash scripts/chup-tc.sh).

Bảng đủ 8 cột theo đề: mã · tiền điều kiện · các bước · dữ liệu thử · kết quả mong đợi · mức ưu tiên · kết quả thực tế · PASS/FAIL. Cột mã ghi kèm chức năng, loại test và quy tắc / yêu cầu được truy vết (bảng truy vết đầy đủ: docs/test-plan.md*). Kết quả thực tế chép từ lần chạy thật (*docs/tc-ket-qua.txt*), không chép lại cột mong đợi.*

Mã · chức năng · loại · truy vết	Tiền điều kiện	Các bước	Dữ liệu thử	Kết quả mong đợi	Ưu tiên	Kết quả thực tế	P/F
TC-01 · Đăng nhập · luồng đúng · *NFR05*	2 tài khoản quoc@mekong.vn (Nhân viên tiếp nhận) và phu@mekong.vn (Quản lý) — dựng trong test	1. bash scripts/tc.sh TC-01 (đăng nhập Nhân viên → đăng xuất → đăng nhập Quản lý)	Mật khẩu đúng của từng tài khoản	Nhân viên được chuyển tới **Tra cứu khách hàng**; Quản lý tới **Danh sách yêu cầu**	Cao	1 runs, 6 assertions, 0 failures – Nhân viên vào Tra cứu khách hàng, Quản lý vào Danh sách yêu cầu	PASS
TC-02 · Đăng nhập · dữ liệu sai · *NFR05*	Tài khoản quoc@mekong.vn tồn tại	1. bash scripts/tc.sh TC-02	Mật khẩu sai-mat-khau	422, ở lại trang đăng nhập, báo "Email hoặc mật khẩu không đúng."; số phiên (Session) KHÔNG tăng	Cao	1 runs, 5 assertions, 0 failures – 422, báo lỗi, số phiên không tăng	PASS
TC-03 · Tra cứu khách hàng · luồng đúng · *UC01, BR01*	Khách KH-900001 "Nguyễn Văn Test", SĐT 0909000111	1. bash scripts/tc.sh TC-03	q = "0909 000 111" (có dấu cách)	Hiện đúng 1 khách "Nguyễn Văn Test", có nút tạo yêu cầu cho khách đó	Cao	1 runs, 3 assertions, 0 failures – tìm thấy đúng "Nguyễn Văn Test"	PASS
TC-04 · Tra cứu khách hàng · luồng ngoại lệ UC08 ·	Như TC-03	1. bash scripts/tc.sh TC-04	0912000111, KH-000999, Không Có Ai	Mỗi lần: khung "không tìm thấy" có lặp lại từ khóa; KHÔNG có khách nào, không có nút tiếp tục	Cao	1 runs, 12 assertions, 0 failures – cả 3 từ khóa đều hiện khung không tìm	PASS

Mã · chức năng · loại · truy vết	Tiền điều kiện	Các bước	Dữ liệu thử	Kết quả mong đợi	Ưu tiên	Kết quả thực tế	P/F
UC08, BR07						thấy, không có khách	
TC-05 · Thêm khách hàng · giá trị biên (trong vùng) · *BR12*	db:reset; server đang chạy; đăng nhập quoc@mekong.vn	1. Đếm khách. 2. POST /api/customers. 3. Đếm lại (bash scripts/tc.sh TC-05)	phone đúng **10** chữ số, bắt đầu bằng 0	201 Created, mã KH-0NNNNN; số khách **+1**	Cao	Trước 310 → HTTP/1.1 201 Created, mã KH-000001, phone 0994316007 → sau 311 (+1)	PASS
TC-06 · Thêm khách hàng · giá trị biên (ngoài vùng) · *BR01*	Như TC-05	1. Đếm khách. 2. POST /api/customers. 3. Đếm lại (bash scripts/tc.sh TC-06)	phone = 09123456 7 (**9** chữ số)	400 DU_LIEU_KHONG_HOP_LE, cac_truong có phone; số khách KHÔNG đổi	Cao	Trước 311 → HTTP/1.1 400 Bad Request, DU_LIEU_KHONG_HOP_LE, phone: "phải gồm 10 chữ số, bắt đầu bằng 0" → sau 311 (không đổi)	PASS
TC-07 · Tạo yêu cầu bảo hành · luồng đúng (BR05, BR06) · *BR01, BR03, BR05, BR06*	Khách KH-900001; nhóm sự cố; mức ưu tiên Cao 24h	1. bash scripts/tc.sh TC-07 (gửi form Màn hình 2)	Điện thoại Samsung Galaxy A55, serial SN-NEW-001, mô tả "Máy không lên nguồn"	Yêu cầu +1, thiết bị +1; mã BH-YYYY-000001; trạng thái **Mới tiếp nhận**; hạn = lúc tạo + 24h; chuyển tới Phiếu tiếp nhận	Cao	1 runs, 13 assertions, 0 failures – mã BH, Mới tiếp nhận, hạn +24h, về Phiếu tiếp nhận	PASS
TC-08 · Tạo yêu cầu bảo hành · dữ liệu sai (BR04) · *BR02, BR03, BR04*	Như TC-07	1. bash scripts/tc.sh TC-08	Bỏ trống **hãng** và **mức ưu tiên**	422; báo lỗi dưới ô hãng và ô mức ưu tiên; số yêu cầu và thiết bị KHÔNG đổi	Cao	1 runs, 10 assertions, 0 failures – 422, báo lỗi hãng và mức ưu tiên, không tạo yêu cầu / thiết bị	PASS
TC-09 · Danh sách yêu cầu · luồng đúng (UC07) · *UC07, BR10*	3 yêu cầu: …000001 còn hạn, …000002 quá hạn, …000003 đã hoàn tất	1. bash scripts/tc.sh TC-09	Bộ lọc status = overdue	Bảng chỉ còn đúng 1 dòng BH-2026-000002	Trung bình	1 runs, 1 assertions, 0 failures – chỉ còn BH-2026-000002	PASS

Mã · chức năng · loại · truy vết	Tiền điều kiện	Các bước	Dữ liệu thử	Kết quả mong đợi	Ưu tiên	Kết quả thực tế	P/F
TC-10 · Cập nhật trạng thái · vi phạm quy tắc (BR08) · *BR08*	Yêu cầu BH-2099-000101 ở **Mới tiếp nhận**	1. bash scripts/tc.sh TC-10	Chuyển thẳng sang completed	Báo "Không thể chuyển…"; trạng thái VẪN là Mới tiếp nhận	Cao	1 runs, 3 assertions, 0 failures – báo không thể chuyển, vẫn Mới tiếp nhận	PASS
TC-11 · Phân quyền · bảo mật (OWASP A01) · *TD-29, OWASP A01*	Đăng nhập **Quản lý**; yêu cầu BH-2099-000901 mô tả "Lỗi"	1. bash scripts/tc.sh TC-11 (mở form sửa → gửi sửa → gửi xóa)	Mô tả mới "Sửa trộm"	Mở form sửa bị chuyển về Danh sách; mô tả VẪN là "Lỗi"; số yêu cầu KHÔNG đổi	Cao	1 runs, 6 assertions, 0 failures – form sửa bị chuyển về Danh sách, mô tả vẫn "Lỗi", số yêu cầu không đổi	PASS
TC-12 · Nạp dữ liệu · tính tái lập · *TD-28*	Test: CSDL test trống. Thực tế: db:reset đã nạp bộ L2	1. bash scripts/tc.sh TC-12 (test tự động + bin/rails import:l2 lần nữa)	data/mau/*.csv	Test PASS; import:l2 báo "mới 0" cho khách, thiết bị, yêu cầu; số KH / TB / YC trước = sau	Cao	Test: 1 runs, 5 assertions, 0 failures. import:l2 lần nữa: nhóm sự cố mới 0, khách hàng mới 0, thiết bị mới 0, yêu cầu mới 0 (loại 7 dòng). KH 311 · TB 31 · YC 30 trước = sau	PASS
TC-13 · Đăng xuất · luồng đúng + bảo mật phiên (A07) · *NFR05, OWASP A07*	Đã đăng nhập quoc@mekong.vn	1. bash scripts/tc.sh TC-13 (bấm Đăng xuất → mở lại Tra cứu khách hàng)	—	Số phiên (Session) **−1**; chuyển về trang đăng nhập; mở lại Tra cứu khách hàng bị chuyển về đăng nhập	Cao	1 runs, 8 assertions, 0 failures – phiên −1, về trang đăng nhập, mở lại Tra cứu bị chuyển về đăng nhập	PASS
TC-14 · Sửa yêu cầu bảo hành · luồng đúng (TD-29) · *TD-29, BR06*	Đăng nhập Nhân viên tiếp nhận; yêu cầu BH-2099-000960 mô tả "Pin chai", model A55	1. bash scripts/tc.sh TC-14 (gửi form sửa)	Mô tả "Pin phồng", model "A56"	Về Phiếu tiếp nhận, báo "Đã cập nhật…"; lịch sử có dòng "Cập nhật thông tin"; CSDL lưu mô tả "Pin phồng", model "A56"	Trung bình	1 runs, 9 assertions, 0 failures – về Phiếu, báo Đã cập nhật, lịch sử có Cập nhật thông tin, lưu "Pin phồng" / A56	PASS

Mã · chức năng · loại · truy vết	Tiền điều kiện	Các bước	Dữ liệu thử	Kết quả mong đợi	Ưu tiên	Kết quả thực tế	P/F
TC-15 · Xóa yêu cầu bảo hành · luồng đúng (TD-29) · *TD-29*	Như TC-14	1. bash scripts/tc.sh TC-15 (bấm Xóa yêu cầu)	Yêu cầu BH-2099-000960	Số yêu cầu **−1**; chuyển về Danh sách yêu cầu; thiết bị SN-GD-1 VẪN còn	Trung bình	1 runs, 6 assertions, 0 failures – số yêu cầu −1, về Danh sách, thiết bị vẫn còn	PASS
TC-16 · Tra cứu khách hàng · bảo mật – chống SQL injection (A05) · *UC01, OWASP A05*	Test: 2 khách KH-900001, KH-900002. Thực tế: db:reset, server đang chạy	1. bash scripts/tc.sh TC-16 (test tự động + curl GET /api/customers?q=…)	' OR '1'='1 và '; DROP TABLE customers; --	Hiện "Không tìm thấy", KHÔNG có khách nào; số khách vẫn 2. API: 200, "data":[], "tong_so":0 dù CSDL có nhiều khách	Cao	Test: 1 runs, 11 assertions, 0 failures. API thật: CSDL có 311 khách, q=' OR '1'='1 → 200, "data":[], "tong_so":0	PASS

Đối chiếu năm loại test BT3

Loại	Test case
Luồng đúng	TC-01, TC-03, TC-07, TC-09, TC-14, TC-15
Giá trị biên – hai bên ranh giới SĐT 10 chữ số	TC-05 (10 chữ số – nhận), TC-06 (9 chữ số – từ chối)
Dữ liệu sai	TC-02 (sai mật khẩu), TC-08 (thiếu trường)
Bảo mật	TC-11 (A01 – Quản lý không sửa / xóa), TC-13 (A07 – phiên bị xóa khi đăng xuất), TC-16 (A05 – SQL injection)
Tái lập	TC-12
Luồng ngoại lệ / vi phạm quy tắc (bổ sung)	TC-04 (UC08), TC-10 (BR08)

MỤC 3. BUG LOG

Mã	Mức	Tóm tắt	Commit
BUG-01	Critical	Quản lý (chỉ được xem) vào được Tra cứu, Tạo yêu cầu và đổi trạng thái	af9543a
BUG-02	Major	Clone sạch thiếu CustomerRowNormalizer, không dựng được hệ thống	717f4c5
BUG-03	Major	Thiếu route /api và Api::WarrantyRequestsController, API không dùng được	ce377e4
BUG-04	Minor	Thẻ thống kê bị cắt dòng mốc so sánh	515d960
BUG-05	Minor	3 test sửa / xóa yêu cầu hỏng do biến @request trùng biến của Rails	a4d69bc
BUG-06	Major	11 gem (Rails, rack, nokogiri…) có 44 lỗ hổng đã công bố, gồm 1 Critical và 5 High	e7c4923
BUG-07	Minor	Gửi JSON hỏng bị trả 500 LOI_HE_THONG thay vì 400	1ac9df3
BUG-08	Minor	Mật khẩu demo bị in ra log container khi nạp dữ liệu mẫu	50d456d
BUG-09	Major	Clone bằng Git for Windows thì container app không khởi động (script bị đổi sang CRLF)	5d8f608

BUG-01 — Quản lý trung tâm dùng được chức năng của Nhân viên tiếp nhận

- **Mã và mức độ:** BUG-01 · CRITICAL — lỗ hổng kiểm soát truy cập (OWASP A01): vai trò chỉ được xem lại tạo và đổi trạng thái được yêu cầu, dữ liệu bị thay đổi thật (theo bảng quyền TD-24 lúc đó).
- **Phát hiện bởi:** phép thử clone vào thư mục trống trước khi nộp BT2; test test/integration/authorization_test.rb (nay TC-11).
- **Triệu chứng:** trên bản clone mới, đăng nhập phu@mekong.vn vẫn mở được Tra cứu khách hàng, Tạo yêu cầu và bấm đổi trạng thái; sidebar không khóa mục nào.
- **Cách tái lập:** clone commit trước af9543a → db:prepare, db:seed → đăng nhập Quản lý → mở /customers/search và PATCH /warranty_requests/:id/update_status — thành công; luôn xảy ra (do thiếu mã, không phụ thuộc dữ liệu).
- **Nguyên nhân:** khi commit TD-24, các dòng require_permission trong CustomersController, WarrantyRequestsController và phần khóa mục trong _sidebar.html.erb chưa được git add; máy phát triển chạy đúng vì còn bản chưa commit, repo thì thiếu.
- **Cách sửa:** commit bổ sung require_permission :lookup_customers, :create_requests, :view_requests, :change_status ở hai controller và sidebar khóa mục theo can?; thêm bước git status --short phải trống trước khi push.
- **Commit và xác nhận:** https://github.com/anquoc2024/cdtn1-thaianquoc-2/commit/af9543a7c40aa7bda883edde3b2c17da7ef72791 · chạy lại authorization_test.rb → PASS; phép thử clone sạch đạt. Hồi quy: TC-11 (Quản lý không sửa, không xóa yêu cầu).

BUG-02 — Clone sạch không dựng được hệ thống vì thiếu CustomerRowNormalizer

- **Mã và mức độ:** BUG-02 · MAJOR — người khác không dựng được hệ thống từ repo; trên máy phát triển vẫn chạy được.
- **Phát hiện bởi:** phép thử clone vào thư mục trống theo README (BT2).
- **Triệu chứng:** bin/rails db:seed và các test nạp CSV dừng với NameError: uninitialized constant CustomerRowNormalizer.
- **Cách tái lập:** clone commit trước 717f4c5 → bundle install → bin/rails db:prepare → lỗi; luôn xảy ra trên bản clone mới.
- **Nguyên nhân:** tệp src/app/services/customer_row_normalizer.rb (làm sạch SĐT, họ tên, email) chưa được git add khi commit TD-23; CustomerCsvImporter gọi lớp này nên mọi lệnh nạp dữ liệu đều hỏng.

- **Cách sửa:** commit src/app/services/customer_row_normalizer.rb (88 dòng).
- **Commit và xác nhận:** https://github.com/anquoc2024/cdtn1-thaianquoc-2/commit/717f4c57ec119762b5a46d7dba88d32065a3a502 · clone lại → db:prepare chạy hết; test/services/customer_row_normalizer_test.rb 5/5 PASS.

BUG-03 — API yêu cầu bảo hành không dùng được: thiếu route và controller

- **Mã và mức độ:** BUG-03 · MAJOR — toàn bộ /api/warranty_requests không dùng được; giao diện web vẫn tạo được yêu cầu (có cách làm khác).
- **Phát hiện bởi:** bin/rails test sau khi thêm API (TD-21) — 12 errors.
- **Triệu chứng:** gọi GET /api/warranty_requests trả 404 trang HTML; test API báo lỗi route / hằng số không tồn tại.
- **Cách tái lập:** checkout commit trước ce377e4 → bin/rails test test/integration/api_warranty_requests_test.rb → 12 errors.
- **Nguyên nhân:** config/routes.rb thiếu namespace :api và tệp app/controllers/api/warranty_requests_controller.rb chưa được commit.
- **Cách sửa:** thêm namespace :api (health, warranty_requests, PATCH status) vào routes.rb và commit controller.
- **Commit và xác nhận:** https://github.com/anquoc2024/cdtn1-thaianquoc-2/commit/ce377e41f1a843949bf8678149b2467993efef7d · chạy lại api_warranty_requests_test.rb → PASS. Hồi quy: TC-05, TC-06 (gọi API thật qua namespace :api).

BUG-04 — Thẻ thống kê bị cắt dòng mốc so sánh

- **Mã và mức độ:** BUG-04 · MINOR — chỉ ảnh hưởng trình bày, số liệu và nghiệp vụ vẫn đúng.
- **Phát hiện bởi:** kiểm tra giao diện Màn hình 3 khi chụp ảnh minh chứng.
- **Triệu chứng:** dưới 4 thẻ thống kê, dòng mốc so sánh ("hôm qua: 0", "trên 14 yêu cầu đang mở"…) bị cắt nửa dưới.
- **Cách tái lập:** đăng nhập → Danh sách yêu cầu → nhìn đáy 4 thẻ; xảy ra mọi lần.
- **Nguyên nhân:** Propshaft nạp CSS theo bảng chữ cái; .stat { height: auto } trong states.css bị .stat { height: 84px; overflow: hidden } trong warranty.css (nạp sau) đè.
- **Cách sửa:** đổi selector thành .stats .stat trong src/app/assets/stylesheets/states.css để có độ ưu tiên cao hơn.
- **Commit và xác nhận:** https://github.com/anquoc2024/cdtn1-thaianquoc-2/commit/515d960d13341264e7b83e68bab02a4443470a47 · Ctrl + Shift + R → 4 thẻ hiện đủ dòng mốc; bin/rails test PASS.

BUG-05 — 3 test sửa / xóa yêu cầu hỏng do biến @request

- **Mã và mức độ:** BUG-05 · MINOR — lỗi ở code test; chức năng sửa / xóa trên giao diện vẫn đúng.
- **Phát hiện bởi:** bin/rails test (Buổi 14) — 128 runs, 2 failures, 1 error.
- **Triệu chứng:** warranty_request_edit_test.rb: mở form sửa ra 404; redirect tới /warranty_requests/#<ActionDispatch::Request…>; NoMethodError: undefined method 'reload' for ActionDispatch::Request.
- **Cách tái lập:** bin/rails test test/integration/warranty_request_edit_test.rb → 3/4 test hỏng, lặp lại ở mọi lần chạy.
- **Nguyên nhân:** test đặt biến @request cho yêu cầu bảo hành, trùng biến nội bộ @request của ActionDispatch::IntegrationTest; sau lệnh get / patch đầu tiên Rails ghi đè thành đối tượng Request.
- **Cách sửa:** đổi @request → @warranty_request trong src/test/integration/warranty_request_edit_test.rb.
- **Commit và xác nhận:** https://github.com/anquoc2024/cdtn1-thaianquoc-2/commit/a4d69bcab84f3a97b37a5a1bca29485653385cd8 · chạy lại tệp test → 4/4

PASS; toàn bộ bin/rails test → 0 failures, 0 errors (ảnh docs/screenshots/BUG-05-fixed.png).

BUG-06 — Thư viện có lỗ hổng bảo mật đã công bố

- **Mã và mức độ:** BUG-06 · MAJOR — 44 lỗ hổng đã công bố trong 11 gem (1 Critical, 5 High), trong đó có loại ứng dụng đang dùng tới: rack multipart DoS (High), Strong Parameters bypass (High), giả IP qua RemoteIp (High). Hệ thống vẫn chạy đúng chức năng và chưa thấy bị khai thác; lỗ hổng Critical nằm ở Active Storage phân tích ảnh – tính năng luồng L2 không dùng – nên xếp Major, không phải Critical.
- **Phát hiện bởi:** checklist bảo mật mục 7 (OWASP A03) — bin/bundler-audit check --update trong scripts/kiem-bao-mat.sh báo Vulnerabilities found! (ảnh docs/screenshots/BUG-06-truoc.png).
- **Triệu chứng:** bundler-audit liệt kê, ví dụ: rack 3.2.7 GHSA-5v64-555j-wjxm (High), rails-html-sanitizer 1.7.1 GHSA-cpgf-5mjf-73p7, nokogiri 1.19.4 GHSA-v995-44v6-5fqm. Đối chiếu đủ với ruby-advisory-db (bản 09/11/2026): actionpack 7 · activestorage 7 · addressable 7 · loofah 6 · rack 4 · actionview 4 · nokogiri 3 · activesupport 2 · rails-html-sanitizer 2 · actionmailbox 1 · crass 1 = 44 mã.
- **Cách tái lập:** checkout commit trước e7c4923 → cd src && bin/bundler-audit check --update → Vulnerabilities found!; luôn xảy ra (phụ thuộc phiên bản trong Gemfile.lock, không phụ thuộc dữ liệu).
- **Nguyên nhân:** Gemfile.lock vẫn khóa các phiên bản lúc tạo dự án; các bản vá bảo mật phát hành sau đó (Rails 8.1.4.1, rack 3.2.8…) chưa được cập nhật vì trước đó chưa có bước kiểm lỗ hổng phụ thuộc.
- **Cách sửa:** bundle update --conservative đúng các gem bị báo — chỉ đổi src/Gemfile.lock: Rails 8.1.4 → 8.1.4.1 (cả nhóm actionpack, actionview, activestorage, activesupport, actionmailbox…), rack 3.2.7 → 3.2.8, nokogiri 1.19.4 → 1.19.7, rails-html-sanitizer 1.7.1 → 1.7.2, loofah 2.25.2 → 2.25.3, crass 1.0.7 → 1.0.8, addressable 2.9.0 → 2.9.2. (Tiêu đề commit chỉ nêu ba gem; nội dung commit nâng đủ 11 gem trên.)
- **Commit và xác nhận:** https://github.com/anquoc2024/cdtn1-thaianquoc-2/commit/e7c49235b1f9d6a09734586af0c617862c3495b7 · bin/bundler-audit check → No vulnerabilities found; bin/importmap audit → No vulnerable packages found (ảnh sec-7.png); bin/rails test → 131 runs, 0 failures; build lại Docker → db, app healthy; CI xanh.

BUG-07 — Gửi JSON hỏng bị trả 500 thay vì 400

- **Mã và mức độ:** BUG-07 · MINOR — lỗi phía người gửi bị coi là sự cố hệ thống: trả 500 LOI_HE_THONG và ghi log mức ERROR (báo động giả cho người vận hành). Không lộ thông tin bên trong, chức năng chính không bị ảnh hưởng.
- **Phát hiện bởi:** checklist bảo mật mục 5a (OWASP A10 – xử lý tình huống bất thường) — gửi thân JSON hỏng tới POST /api/customers (ảnh docs/screenshots/BUG-07-truoc.png).
- **Triệu chứng:** HTTP/1.1 500 Internal Server Error · {"error":"LOI_HE_THONG", … "ma_tra_cuu":"4c57e01f-56c1-4a45-8ddf-7c1cc368cb3d"}.
- **Cách tái lập:** curl -X POST http://localhost:3000/api/customers -H "Authorization: Bearer <token>" -H "Content-Type: application/json" -d '{"name":"A", phone' → 500; luôn xảy ra.
- **Nguyên nhân:** Rails ném ActionDispatch::Http::Parameters::ParseError khi đọc tham số; Api::BaseController chỉ có rescue_from StandardError chung nên lỗi rơi vào nhánh 500.
- **Cách sửa:** thêm rescue_from ActionDispatch::Http::Parameters::ParseError trả 400 DU_LIEU_KHONG_HOP_LE ("Nội dung gửi lên không phải JSON hợp lệ.") trong src/app/controllers/api/base_controller.rb; thêm test hồi quy "JSON hỏng: 400 …" vào src/test/integration/api_crud_test.rb.

- **Commit và xác nhận:** https://github.com/anquoc2024/cdtn1-thaianquoc-2/commit/1ac9df33586f8236c8a7d43f2564de7d819fc28f · test mới PASS; chạy lại checklist mục 5 → HTTP/1.1 400 Bad Request, DU_LIEU_KHONG_HOP_LE (ảnh sec-5.png); bin/rails test → 131 runs, 0 failures.

BUG-08 — Mật khẩu demo bị in ra log container

- **Mã và mức độ:** BUG-08 · MINOR — mật khẩu của tài khoản demo (dữ liệu mẫu, đã ghi trong Deployment Guide) nằm trong log container, trái nguyên tắc "log không ghi mật khẩu" (OWASP A09). Không có dữ liệu thật bị lộ.
- **Phát hiện bởi:** checklist bảo mật mục 6 — docker compose logs app | grep -n 'Mekong@2026' (ảnh docs/screenshots/BUG-08-truoc.png).
- **Triệu chứng:** 2 dòng log: Nhân viên tiếp nhận: quoc@mekong.vn / Mekong@2026 · lấy token qua POST /api/session và dòng tương tự cho phu@mekong.vn.
- **Cách tái lập:** docker compose down -v && docker compose up -d --build → docker compose logs app | grep Mekong@2026 → 2 dòng; luôn xảy ra ở lần khởi động đầu (lúc seed chạy).
- **Nguyên nhân:** db/seeds.rb in tài khoản demo kèm mật khẩu ra màn hình; bin/docker-entrypoint chạy seed khi khởi động nên đầu ra nằm lại trong log container. Lần sửa trước (967bf5a) mới bỏ token, còn sót mật khẩu.
- **Cách sửa:** src/db/seeds.rb chỉ in mật khẩu và token ở môi trường development; môi trường khác chỉ in email kèm chỉ dẫn tới Deployment Guide. Thêm test src/test/integration/seeds_output_test.rb chạy seed và kiểm tra đầu ra không chứa mật khẩu, không chứa token.
- **Commit và xác nhận:** https://github.com/anquoc2024/cdtn1-thaianquoc-2/commit/50d456d655e82e1e110d8b67bc70ff736a95813a · test mới PASS; down -v → up → chạy lại checklist mục 6 → (không có) (ảnh sec-6.png); bin/rails test → 131 runs, 0 failures.

BUG-09 — Clone trên Windows thì container app không khởi động

- **Mã và mức độ:** BUG-09 · MAJOR — nếu repo được clone bằng Git for Windows (mặc định core.autocrlf=true), container app dừng ngay khi khởi động: hệ thống không lên, phép thử máy trắng (C1) thất bại. Có cách làm khác (clone trong WSL / Linux / macOS, hoặc tắt core.autocrlf) nên xếp Major.
- **Phát hiện bởi:** rà soát phép thử máy trắng trên môi trường khác — giả lập máy Windows bằng git clone -c core.autocrlf=true --branch v1.0 … rồi chạy thử src/bin/docker-entrypoint (ảnh docs/screenshots/BUG-09-truoc.png).
- **Triệu chứng:** file src/bin/docker-entrypoint → … with CRLF line terminators; chạy script → /bin/bash: -: invalid option. Đây chính là ENTRYPOINT của image nên container app thoát, không bao giờ healthy.
- **Cách tái lập:** git clone -q -c core.autocrlf=true --branch v1.0 https://github.com/anquoc2024/cdtn1-thaianquoc-2.git x (tag cũ, trước 5d8f608) → x/src/bin/docker-entrypoint echo ok → invalid option; luôn xảy ra khi Git đổi xuống dòng.
- **Nguyên nhân:** repo không có .gitattributes, nên Git for Windows đổi LF → CRLF khi checkout. Dòng shebang thành #!/bin/bash -e\r; kernel truyền tham số -e\r cho bash và bash từ chối tùy chọn lạ. Mọi lần thử trước đều clone trong WSL (giữ LF) nên không lộ lỗi.
- **Cách sửa:** thêm .gitattributes ở gốc repo: * text=auto eol=lf (checkout luôn giữ LF, bất kể core.autocrlf); CSV, PNG, JPG, PDF, DOCX, drawio giữ nguyên từng byte. Deployment Guide thêm một dòng Troubleshooting.
- **Commit và xác nhận:** https://github.com/anquoc2024/cdtn1-thaianquoc-2/commit/5d8f6088f2417ea4a64839222264ac0cd167e204 · clone lại với core.autocrlf=true → file không còn CRLF, chạy script in ok (ảnh

docs/screenshots/BUG-09-sau.png); phép thử máy trắng theo tag v1.0 gắn lại sau bản sửa vẫn đạt.

MỤC 4. DEPLOYMENT GUIDE

Warranty request intake and classification · Track SE · Tag **v1.0** Author: Thái An Quốc (2374802012635) · Repository: https://github.com/anquoc2024/cdtn1-thaianquoc-2

The system runs as two containers started by Docker Compose:

Service	What it is	Reachable from
app	Rails 8.1 web app + JSON API (Ruby 4.0.5, Puma, RAILS_ENV=production)	http://localhost:3000 (port set by APP_PORT)
db	PostgreSQL 18	Internal Compose network only — no host port is published

No Ruby, Node.js or PostgreSQL installation is needed on the host.

1. Requirements

- Linux, macOS, or Windows 10/11 with WSL 2. Tested on Windows + WSL 2 (Ubuntu) with Docker Engine installed inside Ubuntu.
- Docker Engine 25+ (the healthchecks use start_interval) with the Docker Compose plugin v2.20+. Check: `docker version --format '{{.Server.Version}}'` and `docker compose version`.
- Git 2.30+. Check: `git --version`.
- About 3 GB of free disk space (base images, build cache, database volume).
- Port **3000** free on the host (or choose another port with APP_PORT, see section 3).
- Internet access for the first build (downloads ruby:4.0.5-slim, postgres:18 and Ruby gems).

2. Setup steps

Run each command from an empty folder, in this order.

- Clone the release tag:

```
git clone --branch v1.0 https://github.com/anquoc2024/cdtn1-thaianquoc-2.git && cd cdtn1-thaianquoc-2
```

- Create the environment file from the template (the sample values work as-is on a test machine):

```
cp .env.example .env
```

- Build and start everything:

```
docker compose up -d --build
```

The first run takes about 5–10 minutes depending on network speed (measured: 2 min 12 s when the two base images were already downloaded). On the first start the app container automatically creates the databases, loads the schema and loads the sample data — there is no extra command to run. Wait about 30 seconds after the command returns, then go to section 5.

To stop: `docker compose down` (data is kept). To reset to a clean state: `docker compose down -v` (deletes the database volume; the next up loads the sample data again).

3. Environment variables

Defined in .env (copied from .env.example). .env is listed in .gitignore and must never be committed. **Change every value before a real deployment.**

Name	Meaning	Sample value	Required
APP_PORT	Host port that serves the app	3000	No (default 3000)

Name	Meaning	Sample value	Required
DB_NAME	Main database name; Rails also creates DB_NAME_cache, DB_NAME_queue, DB_NAME_cable	smartcrm	Yes
DB_USER	PostgreSQL user, created by the db container on first start	smartcrm	Yes
DB_PASSWORD	Password of DB_USER	doi-mat-khau-nay	Yes
SECRET_KEY_BASE	Key that signs session cookies. Generate a real one with openssl rand -hex 64	placeholder in .env.example	Yes
RAILS_MAX_THREADS	Puma threads = max database connections per process	5	No (default 5)
RAILS_LOG_LEVEL	Log level: debug, info, warn, error	info	No (default info)

Set by docker-compose.yml, not by .env: RAILS_ENV=production and DB_HOST=db (the service name).

4. Sample data

Loaded **automatically** on the first start, while the database volume is still empty: bin/docker-entrypoint runs bin/rails db:prepare, which creates the databases, loads the schema and runs db/seeds.rb. The seed reads the CSV files in data/mau/ (copied into the image at /data/mau). Later restarts only apply pending migrations and do not load the data again. To load the sample data **again on purpose** (re-run test — it must not create duplicates): docker compose exec app bin/rails import:l2 → every count is "mới 0" (0 new) and the totals stay the same (see step 7 in section 5).

Expected records after loading:

Data	Count	Source
Users (demo accounts)	2	db/seeds.rb
Priorities (SLA 72 h / 48 h / 24 h / 8 h)	4	db/seeds.rb
Issue categories	6	data/mau/issue_categories.csv
Customers	310	279 from customers_mau.csv (first 300 rows: 1 duplicate phone merged, 20 invalid rows rejected) + 31 from customers.csv
Devices	31	data/mau/devices.csv
Warranty requests	30 (18 overdue right after loading; the number grows as time passes)	data/mau/tickets.csv (7 invalid rows in the L2 files rejected)

Demo accounts (sample data, not real secrets — passwords are stored as bcrypt hashes):

Email	Password	Role
quoc@mekong.vn	Mekong@2026	Receptionist (Nhân viên tiếp nhận)
phu@mekong.vn	Mekong@2026	Service centre manager (Quản lý trung tâm)

5. Verify it works

#	Command / action	Expected result
1	docker compose ps	smartcrm-l2-app-1 and smartcrm-l2-db-1 both Up … (healthy); the db PORTS column shows only 5432/tcp (not published)
2	curl http://localhost:3000/api/health	{"status":"ok","db":"ok"}
3	docker compose exec app bin/rails runner 'puts "users=#{User.count} customers=#{Customer.count} devices=#{Device.count} requests=#{WarrantyRequest.count}"'	users=2 customers=310 devices=31 requests=30
4	curl -s -X POST http://localhost:3000/api/session -H "Content-Type: application/json" -d '{"email_address":"quoc@mekong.vn","password":"Mekong@2026"}'	HTTP 200, JSON with "token", "role":"receptionist" and the permission list
5	Open http://localhost:3000, sign in as quoc@mekong.vn	Redirected to **Tra cứu khách hàng**; searching 0843145885 shows customer **Bùi Tuấn Huy, KH-114121**
6	docker compose exec -e RAILS_ENV=test app bin/rails db:test:prepare test	131 runs, 591 assertions, 0 failures, 0 errors, 0 skips
7	docker compose exec app bin/rails import:l2, then the command of step 3 again	Bộ dữ liệu L2: nhóm sự cố mới 0 (đã có 6) · khách hàng mới 0 (đã có 31) · thiết bị mới 0 (đã có 31) · yêu cầu mới 0 (đã có 30) · loại 7 dòng; step 3 still prints users=2 customers=310 devices=31 requests=30

Step 6 runs the whole automated test suite inside the container against a separate test database; it does not touch the sample data. Step 7 loads the sample data a second time to show that loading is repeatable without duplicates.

6.Troubleshooting

Symptom	Cause	Fix
env file …/.env not found	Step 2 was skipped	cp .env.example .env, then docker compose up -d --build
Bind for 0.0.0.0:3000 failed: port is already allocated	Another program uses port 3000	Set APP_PORT=3001 in .env, run docker compose up -d, open http://localhost:3001
app stays starting / unhealthy, or exits	Error during start-up	docker compose logs app --tail 50 (every request line carries a request id)
password authentication failed or role … does not exist after editing DB_USER / DB_PASSWORD	The database volume was created with the old values	docker compose down -v, then docker compose up -d (sample data is reloaded)
Pages show old data or no data	Volume from an earlier run	docker compose down -v, then docker compose up -d
permission denied … /var/run/docker.sock (Linux / WSL)	User is not in the docker group	sudo usermod -aG docker $USER, then open a new terminal

Symptom	Cause	Fix
docker: command not found in WSL	Docker is not available in the WSL distro	Enable WSL integration in Docker Desktop, or install Docker Engine inside the distro
app exits with /bin/bash: -: invalid option	Scripts were checked out with Windows line endings (CRLF) by a Git client that ignores .gitattributes	Clone again with git clone --config core.autocrlf=false --branch v1.0 … (the repo's .gitattributes forces LF since BUG-09)

7. Trial deployment on Render

A public copy of the same Docker image runs at https://smartcrm-app.onrender.com, on Render's free plan. It is only for viewing; the reference way to run the system is sections 2–5 above.

Item	Setting
Platform	Render (render.com) · region Singapore · Free plan
Database	Render Postgres 18 smartcrm-db (database smartcrm, user smartcrm)
Web service	smartcrm-app · language Docker · branch main · root directory empty · Dockerfile path ./Dockerfile (the same image as section 2)
Health check path	/api/health

Steps taken:

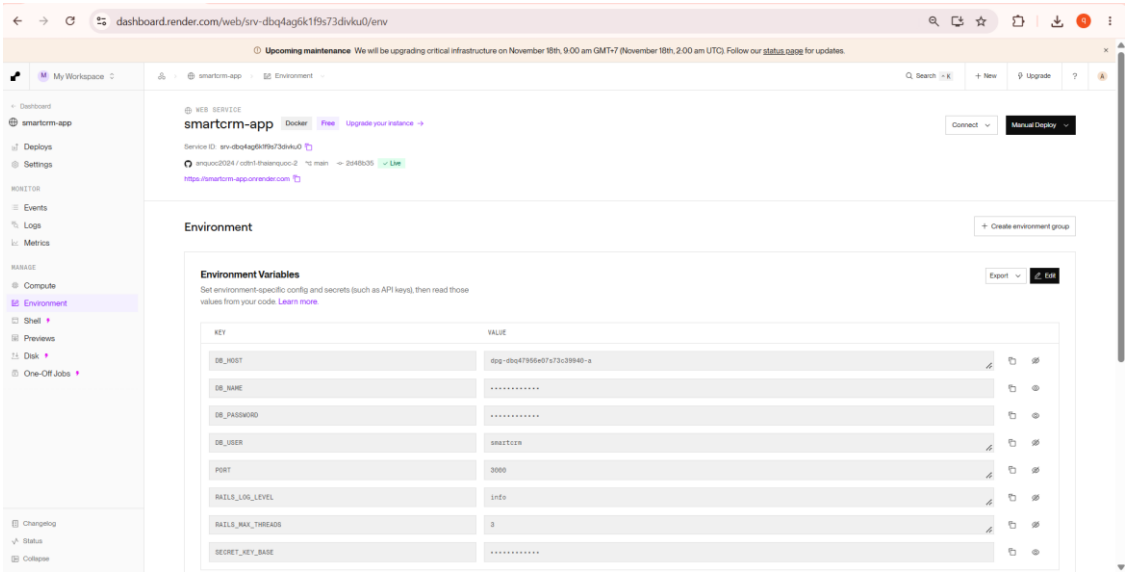
- Render dashboard → **New** → **Postgres**: name smartcrm-db, database smartcrm, user smartcrm, PostgreSQL 18, region Singapore, plan Free. Wait for **Available**, then copy the internal hostname and the password from **Connections**.
- Generate the session key on the local machine: openssl rand -hex 64.
- **New** → **Web Service** → connect the GitHub repository anquoc2024/cdtn1-thaianquoc-2: language Docker, branch main, region Singapore, instance type Free.
- In **Environment**, add the variables in the table below; in **Advanced**, set Health Check Path to /api/health; click **Deploy Web Service**.
- The first deploy builds the image (5–10 minutes). bin/docker-entrypoint runs bin/rails db:prepare, which creates smartcrm plus smartcrm_cache, smartcrm_queue and smartcrm_cable, and loads the sample data. The log ends with Listening on http://0.0.0.0:3000 and the service turns **Live**.
- Check: curl -s https://smartcrm-app.onrender.com/api/health → {"status":"ok","db":"ok"}; sign in with a demo account from section 4; searching 0843145885 shows **Bùi Tuấn Huy, KH-114121**.

Variable	Value on Render
DB_HOST	Internal hostname of smartcrm-db (dpg-…-a)
DB_NAME	smartcrm
DB_USER	smartcrm
DB_PASSWORD	Password of smartcrm-db (secret)
SECRET_KEY_BASE	Output of openssl rand -hex 64 (secret)
PORT	3000 — Puma listens on port 3000 (see Dockerfile)
RAILS_MAX_THREADS	3 — lower than the local default to fit the memory of the free instance
RAILS_LOG_LEVEL	info

RAILS_ENV=production comes from the Dockerfile. The two secrets are entered only in Render's Environment tab and are not in the repository.

Notes:

- The service uses separate `DB_*` variables, as in `docker-compose.yml`, so that `db:prepare` creates the cache, queue and cable databases with their own schemas. Pointing all four connections to one database leaves out the Solid Cache table, and sign-in (rate-limited through Solid Cache) then fails with 500.
- On the free plan the web service sleeps when it receives no traffic for a while; the first request after that can take about a minute.
- A free Render Postgres database expires 30 days after it is created. If it expires, create a new one, update `DB_HOST` and `DB_PASSWORD`, and redeploy — `db:prepare` loads the sample data again.



*Hình 1. Render – service smartcrm-app (Docker, Free, nhánh main, commit 2d48b35, Live): 8 biến môi trường, giá trị bí mật đã che (*docs/screenshots/deploy-render-env.png*)*

MỤC 5. CHECKLIST BẢO MẬT 7 MỤC ĐÃ TỰ ĐÁNH GIÁ

Kiểm bằng lệnh chạy thật trên gói Docker (script scripts/kiem-bao-mat.sh, kết quả chữ docs/sec-ket-qua.txt). Ghi "CHƯA LÀM" ở đúng phần chưa làm. Quá trình kiểm tìm ra 3 bug thật (BUG-06, 07, 08), đã sửa và chạy lại.

#	Mục	OWASP	Trạng thái	Minh chứng (lệnh đã chạy và kết quả · ảnh trong docs/screenshots/)
1	Không có secret nào trong repo	A02	**ĐÃ LÀM**	git ls-files và git log --all -- .env src/config/master.key → rỗng (chưa từng commit); .gitignore chặn .env, master.key; trong image: ls /rails/.env → *No such file*. sec-1.png
2	Dữ liệu vào được kiểm ở tầng ứng dụng	A05	**ĐÃ LÀM**	GET /api/customers?q=' OR '1'='1 → tong_so: 0; SĐT 9 chữ số → 400 DU_LIEU_KHONG_HOP_LE; SQL chỉ ghép hằng số, mọi truy vấn khác có tham số hóa. TC-06, TC-08, TC-16 PASS. sec-2.png
3	Kiểm soát truy cập	A01	**ĐÃ LÀM** (theo vai trò)	Không token → 401; Quản lý sửa / xóa / đổi trạng thái → 403 KHONG_CO_QUYEN ×3; TC-11 PASS. Theo *chủ sở hữu*: **không áp dụng** – luồng L2 không có tài khoản khách hàng, nhân viên dùng chung một hàng đợi (TD-29). sec-3.png
4	Mật khẩu và dữ liệu nhạy cảm được băm / mã hóa	A04, A07	**ĐÃ LÀM** (mật khẩu) · **CHƯA LÀM** (API token)	password_digest = $2a$12$… (bcrypt, 60 ký tự). api_token lưu nguyên văn, không có hạn dùng → kế hoạch: lưu băm SHA-256, thêm hạn dùng (ví dụ 30 ngày); hiện giảm rủi ro bằng DELETE /api/session thu hồi token. sec-4.png
5	Thông báo lỗi không lộ cấu trúc bên trong	A10, A02	**ĐÃ LÀM**	404 → {"status":404,"error":"Not Found"}; tắt CSDL → 503 CSDL_KHONG_KHA_DUNG kèm mã tra cứu, **0** dòng chứa PG::, chuỗi kết nối, đường dẫn mã. JSON hỏng trả 500 → **BUG-07** đã sửa → 400. BUG-07-truoc.png, sec-5.png
6	Nhật ký không ghi secret / dữ liệu cá nhân	A09	**ĐÃ LÀM**	Token thật xuất hiện **0** lần; dòng có SĐT **0**; password, q ghi [FILTERED]. Seed in mật khẩu demo ra log → **BUG-08** đã sửa → (không có). BUG-08-truoc.png, sec-6.png
7	Phụ thuộc ghim phiên bản, đã kiểm lỗ hổng	A03	**ĐÃ LÀM**	Gemfile.lock, importmap.rb đã commit; bundler-audit báo 44 lỗ hổng ở 11 gem → **BUG-06** đã sửa → No vulnerabilities found; importmap audit sạch; brakeman 3 cảnh báo mức Weak, đã đánh giá. BUG-06-truoc.png, sec-7.png

Bản đầy đủ trong repo: docs/checklist-bao-mat.md *(tag* v1.0*) – mỗi mục có đoạn minh chứng chi tiết*

MỤC 6. TÀI LIỆU KỸ THUẬT THEO CHUYÊN NGÀNH – API DOCS (SE)

API documentation in English for **all 21 endpoints** of the JSON API, seven parts each. Base URL http://localhost:3000/api; JSON bodies with Content-Type: application/json. Authentication: Authorization: Bearer <token> from POST /api/session (or the web session cookie plus X-Requested-With: XMLHttpRequest). Every error has the shape {"error": "CODE", "message": "…", "chi_tiet": {…}} and never contains stack traces, SQL or connection strings. "Real example #n" = request *n* in docs/api-examples.md, generated by docs/scripts/goi-api-docker.sh on the Docker stack (46 requests, all returned the expected status). * = required. Full version with one 7-part table per endpoint: docs/api-docs.md (tag v1.0).

1. Method & path · 2. Description	3. Auth / role · 4. Parameters & body	5. Success · 6. Error codes	7. Limits · real example
POST /api/session Sign in; returns API token, role, permissions	*Auth:* None *Params:* email_address*,* password	*OK:* 200 data.token, data.user *Errors:* 401 SAI_THONG_TIN_DANG_NHAP · 429 · 400	10 attempts / 3 min; bcrypt *Example:* #1, #2
DELETE /api/session Sign out: replaces the token	*Auth:* Bearer, any role *Params:* —	*OK:* 204 *Errors:* 401 CHUA_XAC_THUC	Old token invalid at once *Example:* #45, #46
GET /api/health App and database status (Docker healthcheck)	*Auth:* None *Params:* —	*OK:* 200 {"status":"ok","db":"ok"} *Errors:* 503 degraded + ma_tra_cuu	No user data *Example:* #3
GET /api/customers List; search by phone, code, name (UC01)	*Auth:* Bearer · lookup_customers (both roles) *Params:* q, page	*OK:* 200 data[], meta *Errors:* 400 · 401 · 403	20 / page; search ≤ 20 *Example:* #5, #6
GET /api/customers/{id} Customer detail + request count	*Auth:* Bearer · lookup_customers *Params:* id	*OK:* 200 data *Errors:* 404 · 401 · 403	— *Example:* #7, #14
POST /api/customers Create; code KH-0NNNNN generated (BR12)	*Auth:* Bearer · manage_customers (receptionist) *Params:* name*,* phone (10 digits, unique), email, address	*OK:* 201 + Location *Errors:* 400 · 409 TRUNG_DU_LIEU · 403	Codes up to KH-099999 *Example:* #8–#11
PATCH /api/customers/{id} Update the fields sent	*Auth:* Bearer · manage_customers *Params:* name, phone, email, address	*OK:* 200 data *Errors:* 400 · 409 · 404 · 403	code cannot change *Example:* #12
DELETE /api/customers/{id} Delete a customer with no device	*Auth:* Bearer · manage_customers *Params:* id	*OK:* 204 *Errors:* 409 DANG_DUOC_SU_DUNG · 404 · 403	BR13 *Example:* #13, #15
GET /api/issue_categories Categories with usage count	*Auth:* Bearer · view_requests *Params:* —	*OK:* 200 data[], meta.tong_so *Errors:* 401 · 403	Not paginated *Example:* #16

1. Method & path · 2. Description	3. Auth / role · 4. Parameters & body	5. Success · 6. Error codes	7. Limits · real example
GET /api/issue_categories/{id} Category detail	*Auth:* Bearer · view_requests *Params:* id	*OK:* 200 data *Errors:* 404 · 401 · 403	— *Example:* #17
POST /api/issue_categories Create a category	*Auth:* Bearer · manage_catalog (receptionist) *Params:* name* (unique, ≤ 100), description	*OK:* 201 + Location *Errors:* 400 · 409 TRUNG_DU_LIEU · 403	— *Example:* #18, #19
PATCH /api/issue_categories/{id} Update a category	*Auth:* Bearer · manage_catalog *Params:* name, description	*OK:* 200 data *Errors:* 400 · 409 · 404 · 403	— *Example:* #20
DELETE /api/issue_categories/{id} Delete an unused category	*Auth:* Bearer · manage_catalog *Params:* id	*OK:* 204 *Errors:* 409 DANG_DUOC_SU_DUNG · 404 · 403	BR14 *Example:* #21, #22
GET /api/priorities Priority levels and SLA hours (BR06)	*Auth:* Bearer · view_requests *Params:* —	*OK:* 200 data[] *Errors:* 401 · 403	Read-only *Example:* #23
GET /api/stats Dashboard figures with baselines (UC07)	*Auth:* Bearer · view_request_list *Params:* —	*OK:* 200 data *Errors:* 401 · 403	Computed per call *Example:* #24
GET /api/warranty_requests Filter, sort, paginate requests (UC07)	*Auth:* Bearer · view_request_list *Params:* q, status, issue_category_id, priority_id, from, to, sort, page	*OK:* 200 data[], meta *Errors:* 401 · 403	8 / page *Example:* #4, #25
GET /api/warranty_requests/{id} Detail, allowed next statuses, history	*Auth:* Bearer · view_requests *Params:* id	*OK:* 200 data *Errors:* 404 · 401 · 403	— *Example:* #26, #27
POST /api/warranty_requests Create (UC06): code BH-YYYY-NNNNNN, status received, due_at = now + SLA	*Auth:* Bearer · create_requests (both roles) *Params:* customer_id, device_type, brand, model, serial_number, purchase_date, issue_description, issue_category_id, priority_id*	*OK:* 201 + Location *Errors:* 400 (fields, bad JSON) · 404 KHACH_HANG_KHONG_TON_TAI · 409 SERIAL_THUOC_KHACH_KHAC	One transaction *Example:* #28–#32
PATCH /api/warranty_requests/{id}	*Auth:* Bearer · edit_requests (receptionist)	*OK:* 200 data *Errors:* 400 · 403 · 404	Serial, customer, status fixed

1. Method & path · 2. Description	3. Auth / role · 4. Parameters & body	5. Success · 6. Error codes	7. Limits · real example
Update fields; new priority recalculates due_at (TD-29)	*Params:* device fields, issue_description, issue_category_id, priority_id		*Example:* #33–#35
DELETE /api/warranty_requests/{id} Delete request and history (TD-29)	*Auth:* Bearer · delete_requests (receptionist) *Params:* id	*OK:* 204 *Errors:* 403 · 404 · 409 KHONG_XOA_DUOC	Cannot be undone *Example:* #42–#44
PATCH /api/warranty_requests/{id}/status Change status (BR08, BR09)	*Auth:* Bearer · change_status (receptionist) *Params:* status*, note (required to cancel)	*OK:* 200 data *Errors:* 400 · 400 TRANG_THAI_KHONG_HOP_LE · 400 THIEU_LY_DO_HUY · 422 CHUYEN_TRANG_THAI_KHONG_HOP_LE · 409 YEU_CAU_DA_DONG	No skipping; closed stays closed *Example:* #36–#41

Real examples (from docs/api-examples.md, shortened: "…" marks fields left out; tokens replaced by placeholders)

Example #1 – sign in (200)

```
curl -X POST -H 'Content-Type: application/json' \
    -d '{"email_address":"quoc@mekong.vn","password":"Mekong@2026"}' \
    'http://localhost:3000/api/session'
HTTP 200  {"data": {"token": "<token>", "token_type": "Bearer",
         "user": {"id": 1, "full_name": "Thái An Quốc", "email_address": "quoc@mekong.vn",
                "role": "receptionist", "role_label": "Nhân viên tiếp nhận",
                "permissions": ["lookup_customers", "create_requests", "change_status", …]}}}
```

Example #28 – create a warranty request (201)

```
curl -X POST -H 'Authorization: Bearer <token_receptionist>' \
    -H 'Content-Type: application/json' \
    -d '{"customer_id":280,"device_type":"Điện thoại","brand":"Samsung","model":"Galaxy S24",
        "serial_number":"SN-API-1794322248","purchase_date":"2026-05-01",
        "issue_description":"Màn hình chớp tắt","issue_category_id":1,"priority_id":3}' \
    'http://localhost:3000/api/warranty_requests'
HTTP 201  location: /api/warranty_requests/31
{"data": {"id": 31, "code": "BH-2026-000031", "status": "received",
        "status_label": "Mới tiếp nhận", "overdue": false, "priority": "Cao", "sla_hours": 24,
        "customer_code": "KH-200001", …, "created_at": "2026-11-10T21:50:52+07:00",
        "due_at": "2026-11-11T21:50:52+07:00", …, "allowed_next_statuses": ["in_progress", "cancelled"], …}}
```

Example #36 – skipping a status step (422) · Example #32 – malformed JSON (400)

```
curl -X PATCH -H 'Authorization: Bearer <token_receptionist>' \
    -H 'Content-Type: application/json' -d '{"status":"completed"}' \
    'http://localhost:3000/api/warranty_requests/31/status'
HTTP 422  {"error": "CHUYEN_TRANG_THAI_KHONG_HOP_LE",
         "message": "Không thể chuyển từ "Mới tiếp nhận" sang "Hoàn tất"."}
```

```
curl -X POST -H 'Authorization: Bearer <token_receptionist>' \
    -H 'Content-Type: application/json' -d '{"customer_id": 1, brand' \
    'http://localhost:3000/api/warranty_requests'
HTTP 400  {"error": "DU_LIEU_KHONG_HOP_LE", "message": "Nội dung gửi lên không phải JSON hợp lệ."}
```

MỤC 7. ẢNH CHỤP MINH CHỨNG

Ảnh chụp màn hình lần chạy thật trên máy trắng: clone theo tag v1.0 (commit 22d2d94) vào thư mục trống, xóa sạch container, volume, image và bộ nhớ đệm build trước đó, rồi làm nguyên văn Deployment Guide. Ảnh từng test case (TC-01 → TC-16), từng mục checklist và ảnh trước / sau của mỗi bug nằm trong docs/screenshots/ của repo.

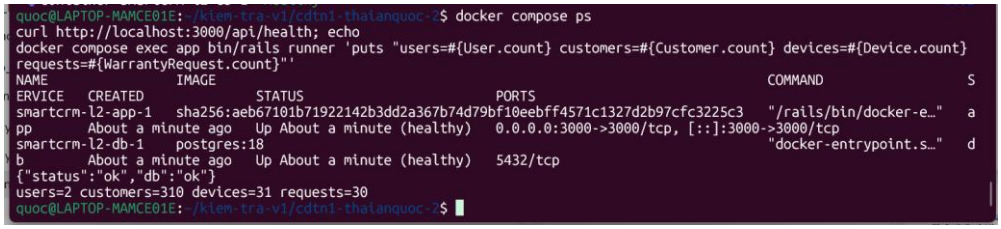
7.1. Clone theo tag v1.0 vào thư mục trống



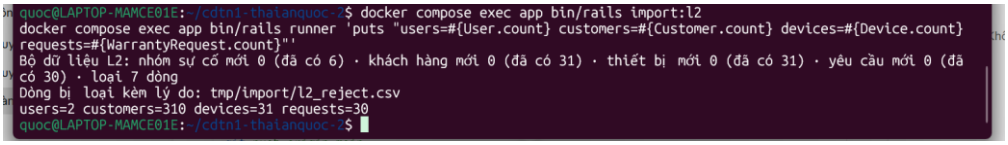
Hình 1. git clone --branch v1.0*: đúng tag, commit* 22d2d94*; có .env.example và .gitattributes, chưa có .env (*docs/screenshots/may-trang-v1-1-clone.png*)*

7.2. Hệ thống lên được bằng hai lệnh, có dữ liệu mẫu

Chạy cp .env.example .env rồi docker compose up -d --build: image được build lại từ đầu trong 199 giây, db Healthy, app Started (ảnh docs/screenshots/may-trang-v1-build.png). Sau khoảng 30 giây:

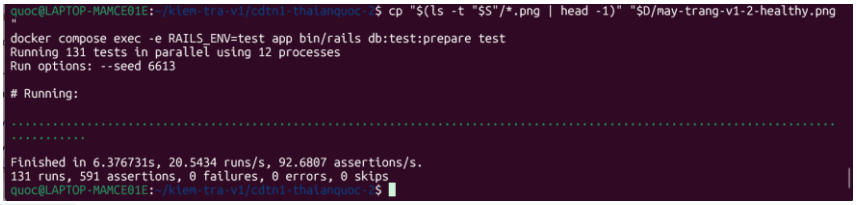


*Hình 2. Container db, app healthy; /api/health ok; 2 người dùng, 310 khách hàng, 31 thiết bị, 30 yêu cầu (*docs/screenshots/may-trang-v1-2-healthy.png*)*



*Hình 3. Nạp dữ liệu lần thứ hai: mọi mục "mới 0", tổng số bản ghi không đổi (C8) (*docs/screenshots/docker-import-lan-2.png*)*

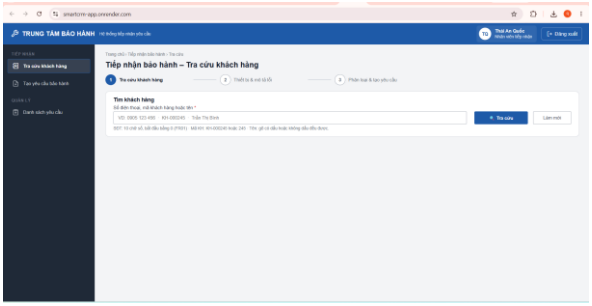
7.3. Bộ kiểm thử PASS bằng một lệnh



Hình 4. bin/rails test *trong container của bản clone theo tag v1.0: 131 runs, 591 assertions, 0 failures, 0 errors (*docs/screenshots/may-trang-v1-3-test.png*)*

Ngoài ra, CI GitHub Actions chạy toàn bộ test trên máy sạch cũng xanh (ảnh docs/screenshots/ci-pass.png).

7.4. Bản deploy thử (điểm cộng)



Hình 5. https://smartcrm-app.onrender.com *– đăng nhập bằng tài khoản demo, màn hình Tra cứu khách hàng (*docs/screenshots/deploy-render.png*)*

PHỤ LỤC. BẢNG KHAI BÁO SỬ DỤNG CÔNG CỤ AI (GIAI ĐOẠN 3)

Công cụ	Dùng vào việc gì	Áp dụng ở tệp / hàm nào	Em đã kiểm chứng thế nào
Trợ lý AI dạng chat (ChatGPT)	Gợi ý test case và cách viết một mục Bug Log đủ 7 dòng	docs/test-report.md, docs/bug-log.md	Chỉ giữ test case em tự chạy được trên hệ thống thật; kết quả thực tế ghi từ log chạy thật; nguyên nhân của từng bug em đối chiếu với mã nguồn và bản sửa
Trợ lý AI dạng chat (Claude)	Chọn 16 test case theo chức năng, viết script chạy và chụp từng test case; viết test chống SQL injection	docs/test-plan.md, docs/test-report.md, scripts/tc.sh, scripts/chup-tc.sh, src/test/integration/customer_lookup_test.rb	Tự chạy cả 16 test case, chụp 16 ảnh docs/screenshots/TC-xx.png; điền P/F theo docs/tc-ket-qua.txt; bin/rails test 131 runs, 0 failures
Trợ lý AI dạng chat (Claude)	Tìm nguyên nhân bug, viết bản sửa và test hồi quy	Api::BaseController (BUG-07), db/seeds.rb + seeds_output_test.rb (BUG-08), Gemfile.lock (BUG-06), .gitattributes (BUG-09), warranty_request_edit_test.rb (BUG-05)	Tự tái lập lỗi trước khi sửa (ảnh BUG-0x-truoc.png), chạy lại test sau khi sửa; mỗi bug một commit fix(...) riêng, mở lại link commit để kiểm
Trợ lý AI dạng chat (Claude)	Viết gói Docker và Deployment Guide tiếng Anh	Dockerfile, docker-compose.yml, .dockerignore, .env.example, src/bin/docker-entrypoint, src/config/database.yml, docs/deployment-guide.md	Tự cài Docker Engine trong WSL, chạy phép thử máy trắng theo tag v1.0-rc1 và v1.0; mọi số liệu trong Guide (thời gian build, số bản ghi, kết quả test) lấy từ lần chạy thật
Trợ lý AI dạng chat (Claude)	Viết script tự kiểm 7 mục checklist bảo mật	scripts/kiem-bao-mat.sh, docs/checklist-bao-mat.md	Tự chạy script, chụp ảnh từng mục; phát hiện và sửa BUG-06, 07, 08; ghi trung thực phần chưa làm (API token không hết hạn)
Trợ lý AI dạng chat (Claude)	Viết script gọi 46 request và tài liệu API tiếng Anh	docs/scripts/goi-api-docker.sh, docs/api-examples.md, docs/api-docs.md	Tự chạy script trên Docker (46/46 đúng mã HTTP); ví dụ trong tài liệu chép nguyên từ kết quả chạy; kiểm tra tệp không lộ token

Tôi đã đọc, hiểu và chịu trách nhiệm về toàn bộ nội dung nộp.

Họ và tên: Thái An Quốc **MSSV:** 2374802012635 **Ngày:** 12/11/2026